Étude approfondie de l'architecture des SIEM : Choix techniques, origines historiques et compromis

Étude préparatoire au projet Smart SIEM

20 juin 2026

Introduction

Un Système de Gestion et d'Analyse des Événements de Sécurité (SIEM) n'est pas un produit monolithique né du génie d'un seul concepteur. C'est un assemblage de briques logicielles, chacune ayant été imaginée, testée et parfois réinventée pour répondre à des défis opérationnels rencontrés sur le terrain par les équipes de sécurité. Contrairement à une architecture purement théorique, les choix qui ont présidé à la construction des SIEM modernes sont le fruit de compromis entre performance, fiabilité, coût, et surtout, capacité à faire face à une menace en constante évolution. Cette étude se propose de décortiquer ces choix, en expliquant non seulement ce qui a été fait, mais surtout pourquoi et comment ces décisions ont été prises, souvent en réaction à des échecs ou des limites des solutions précédentes. Nous verrons ainsi que l'architecture du SIEM est un palimpseste, où chaque couche de fonctionnalité a été ajoutée pour combler une faille de la couche inférieure.

1 1. Le module de collecte : pourquoi des agents plutôt que de simples protocoles ?

1.1 Le contexte initial : la simplicité illusoire du Syslog

Dans les années 1990, la collecte de logs se faisait principalement via le protocole Syslog, souvent en mode UDP. Ce protocole, simple et largement supporté par les équipements réseau et les systèmes Unix, permettait d'envoyer des messages texte vers un serveur central. À première vue, cette approche répondait au besoin : pas d'installation, pas d'agent, un serveur qui écoute sur le port 514. Pourtant, cette simplicité cachait des problèmes majeurs.

1.2 Les problèmes rencontrés

1. **Pertes de données :** UDP étant non fiable, en cas de saturation du réseau ou du serveur récepteur, les paquets étaient simplement perdus, sans aucun mécanisme de reprise. Dans un contexte de sécurité, perdre des logs équivaut à perdre des preuves potentielles.

2. **Surcharge du serveur central :** Le traitement des logs (parse, normalisation, stockage) s'effectuait sur le serveur central. Avec la montée en puissance des infrastructures, le nombre de logs à traiter par seconde est devenu tel que le serveur unique n'a pas pu suivre, entraînant des goulets d'étranglement et des défaillances.
3. **Formats propriétaires :** Chaque équipement (Cisco, Check Point, Windows) produisait des logs dans un format différent. Le serveur central devait connaître et parser chacun de ces formats, ce qui complexifiait le développement et limitait l'interopérabilité.
4. **Accès aux sources de données profondes :** Syslog ne permettait pas de collecter les journaux d'événements Windows (Event Log) ou les logs d'applications métier qui ne les écrivaient pas dans des fichiers standard.

1.3 La solution retenue : l'agent de collecte

Face à ces limites, les éditeurs ont inventé l'agent de collecte (ou forwarder) : un petit logiciel installé sur chaque machine source. Pourquoi ce choix ?

— **Déchargement du traitement :** L'agent peut effectuer un prétraitement local : filtrer les événements inutiles, normaliser les formats (par exemple en CEF ou LEEF), et compresser les données avant de les envoyer. Cela réduit considérablement la charge du collecteur central.

— **Fiabilité et résilience :** L'agent peut mettre en cache les logs en cas de panne réseau et les renvoyer ultérieurement, garantissant une perte zéro. Il peut également utiliser des protocoles TCP ou des files d'attente pour assurer une livraison fiable.

— **Accès à des sources variées :** L'agent peut lire les journaux d'événements Windows via WMI, les fichiers d'application, les bases de données, ou même interroger des API. Il devient un couteau suisse de la collecte.

— **Sécurité :** Dans des environnements cloisonnés, l'agent peut initier une connexion sortante vers le collecteur, évitant ainsi d'ouvrir des ports entrants vulnérables sur les machines.

1.4 Le compromis et les alternatives

Le choix de l'agent a introduit une complexité de déploiement et de maintenance (gestion des mises à jour, consommation de ressources, configuration). D'autres approches ont été explorées :

— **Les protocoles de transport fiables :** Certains ont proposé d'utiliser Syslog sur TCP ou même des protocoles comme AMQP (Advanced Message Queuing Protocol) pour assurer une livraison fiable. Mais ces protocoles ne résolvent pas les problèmes de format et de charge.

— **Les collecteurs réseau passifs :** Des solutions comme les sondes réseau capturent les paquets et en extraient les logs, mais elles sont coûteuses et ne permettent pas d'accéder aux logs applicatifs.

— **L'approche *agentless* :** Aujourd'hui, pour les environnements cloud ou les machines virtuelles, on utilise des API (comme AWS CloudTrail, Azure Activity Logs) qui fournissent des logs structurés sans agent, mais cela suppose que le fournisseur cloud offre cette fonctionnalité. Dans les environnements on-premise historiques, l'agent est resté la solution la plus pragmatique.

2 2. Le module de normalisation : l'invention d'un langage commun

2.1 Le problème : la tour de Babel des logs

Les agents ramenaient des données, mais dans des formats totalement disparates : XML, JSON, CSV, formats propriétaires, ou simple texte brut. Un événement de connexion réussie pouvait être représenté par « User logged in », « Authentication success », ou un code comme « 4624 ». Un analyste ne pouvait pas corréler efficacement ces événements sans un effort de traduction manuel, ce qui rendait la détection de menaces inter-sources presque impossible.

2.2 La solution : les schémas de normalisation

Les éditeurs historiques ont proposé des schémas de normalisation :

— **ArcSight** a créé le **Common Event Format (CEF)** en 2009. CEF définit un en-tête avec des champs fixes (version, device, severity, etc.) et une partie extension contenant des paires clé-valeur pour des champs spécifiques (src, dst, msg, etc.).

— **IBM QRadar** a proposé le **Log Event Extended Format (LEEF)** qui suit une logique similaire.

2.2.1 Pourquoi ce choix et pas un autre ?

— **Interopérabilité :** Les équipementiers pouvaient intégrer directement la génération de logs au format CEF/LEEF, facilitant l'intégration. Cela a créé un écosystème.

— **Performance :** Les formats sont conçus pour être parsés efficacement par les moteurs de corrélation, sans avoir besoin d'interpréter du XML ou du JSON complexe.

— **Extensibilité :** La partie extension permet d'ajouter des champs spécifiques sans casser le format de base.

2.3 L'évolution vers les schémas ouverts

Aujourd'hui, des initiatives comme l'**Elastic Common Schema (ECS)** proposent un schéma ouvert, extensible, et compatible avec la stack ELK. ECS standardise les noms de champs et les types, facilitant l'intégration entre différents outils open-source. Le choix des schémas ouverts s'explique par la volonté d'éviter le verrouillage propriétaire et de favoriser la collaboration.

3 3. Le moteur de corrélation : de la règle statique à l'intelligence

3.1 Le problème initial : le bruit des IDS

Les premiers systèmes de détection d'intrusion (IDS) généraient des alertes en abondance, avec un taux de faux positifs très élevé. Les analystes étaient submergés. De plus,

ces systèmes ne pouvaient pas corréler des événements entre différentes sources. Une attaque en plusieurs étapes (reconnaissance, exploitation, installation de backdoor) produisait des alertes isolées sur différents systèmes, sans qu'aucun outil ne fasse le lien.

3.2 La solution : le moteur de corrélation basé sur des règles

Les SIEM ont introduit un moteur de corrélation capable de :

1. **Agréger des événements :** Compter le nombre d'occurrences d'un événement dans une fenêtre temporelle (ex : 5 échecs de connexion en 60 secondes). C'est la règle de seuil.
2. **Reconnaître des séquences :** Détecter une chaîne d'événements dans un ordre précis (ex : échec de connexion, puis connexion réussie, puis téléchargement de fichier).
3. **Corréler inter-sources :** Relier un log de pare-feu (port scanné) avec un log d'Active Directory (authentification inhabituelle).

3.2.1 Pourquoi ce choix ?

Les règles de corrélation sont faciles à définir, à maintenir et à comprendre par les analystes. Elles reproduisent le raisonnement humain de manière automatisée. La fenêtre temporelle glissante permet de capturer des attaques qui s'étalent dans le temps. Le concept de *confidence score* permet de hiérarchiser les alertes.

3.3 Les limites et les évolutions

Les règles statiques ne peuvent pas détecter ce qu'elles ne connaissent pas. Pour remédier à cela, des approches de corrélation basées sur des modèles statistiques ou d'apprentissage automatique ont été explorées, mais elles sont encore moins explicables et plus difficiles à déployer. Le SIEM moderne conserve un moteur de règles solide et l'enrichit avec des flux de renseignement sur les menaces (Threat Intelligence) pour rester à jour face aux nouvelles attaques.

4 4. Le module de stockage : le défi du volume et de la vitesse

4.1 Le problème : les bases relationnelles s'effondrent

Les premières solutions SIEM stockaient les logs dans des bases de données relationnelles (SQL). Avec la montée en puissance des infrastructures (des milliers d'événements par seconde), ces bases sont devenues des goulots d'étranglement. Les requêtes de recherche sur de longues périodes prenaient des minutes, voire des heures. La scalabilité était impossible.

4.2 La solution : l'indexation distribuée et la recherche en texte intégral

— **Splunk** a popularisé l'approche de l'indexation distribuée : les données sont indexées en temps réel dans un moteur de recherche (Splunk Indexer), qui crée des index inversés pour permettre des recherches quasi-instantanées.

— **Elasticsearch** a repris ce principe en open-source, avec une architecture distribuée (sharding, réplication) et des capacités d'agrégation.

4.2.1 Pourquoi ce choix ?

L'indexation en texte intégral permet de rechercher n'importe quel champ sans avoir à définir un schéma rigide à l'avance. La distribution permet de traiter des volumes massifs (plusieurs téraoctets par jour). Les politiques de rétention (30 jours, 6 mois, 1 an) sont gérées par des mécanismes de rollover et de suppression.

4.3 Le compromis : le coût du stockage

Les index ont un coût de stockage élevé (multiplication par 10 de la taille des données brutes). Les organisations doivent trouver un équilibre entre la durée de rétention et le coût, souvent en déplaçant les données plus anciennes vers des stockages moins chers (cold storage) tout en conservant les index pour les données récentes.

5 5. Le module d'alertes et de réponse (SOAR) : de la notification à l'action

5.1 Le problème : le délai de réponse

Les premiers SIEM alertaient simplement les analystes (email, SMS). Mais l'analyste devait ensuite se connecter à différentes consoles (pare-feu, Active Directory, etc.) pour prendre des mesures (bloquer une IP, désactiver un compte). Ce délai de quelques minutes était souvent fatal : l'attaquant avait déjà exfiltré les données ou installé un ransomware.

5.2 La solution : l'orchestration et l'automatisation

Les plateformes SOAR (Security Orchestration, Automation, and Response) ont été intégrées aux SIEM. Elles permettent de définir des *playbooks* : des séquences d'actions automatisées.

— **Blocage IP** : via une API sur le pare-feu.

— **Désactivation de compte** : via une API sur l'annuaire Active Directory.

— **Isolation de machine** : via un outil de gestion de parc (ex : Endpoint Detection and Response).

5.2.1 Pourquoi ce choix ?

L'automatisation réduit le temps de réponse de quelques minutes à quelques secondes. Elle décharge l'analyste des tâches répétitives, lui permettant de se concentrer sur l'analyse. L'approche en playbooks est flexible et peut être adaptée à chaque organisation.

5.3 Les précautions nécessaires

L'automatisation comporte des risques : un faux positif pourrait entraîner le blocage d'un utilisateur légitime ou d'une infrastructure critique. C'est pourquoi les playbooks intègrent souvent une phase de confirmation (mode *confirm*) pour les actions à fort impact, ou utilisent un *confidence score* avant de déclencher l'action.

6 6. Le module de visualisation : adapter l'information au public

6.1 Le problème : un tableau de bord unique ne convient à personne

Les premières interfaces SIEM affichaient des listes d'alertes et des graphiques basiques, souvent conçues par des ingénieurs pour des ingénieurs. Un RSSI (responsable de la sécurité) avait besoin d'une vue synthétique des menaces, tandis qu'un analyste avait besoin de détails techniques. Un auditeur avait besoin de preuves de conformité.

6.2 La solution : des vues par profil

Les SIEM modernes proposent des tableaux de bord personnalisables :

- **Vue Analyste :** focus sur les alertes, les flux suspects, les timelines, les indicateurs techniques.
- **Vue RSSI :** indicateurs clés (nombre d'incidents critiques, temps de réponse, conformité), souvent avec des couleurs de statut (vert/orange/rouge).
- **Vue Auditeur :** rapports de conformité, preuves d'archivage, journaux d'accès.
- **Vue *Crisis Room* :** plein écran, rafraîchissement rapide, pour les salles de crise.

6.2.1 Pourquoi ce choix ?

L'information doit être présentée de manière à être immédiatement comprise par son destinataire, sans effort de traduction. Un tableau de bord bien conçu permet de prendre des décisions en quelques secondes, ce qui est critique en situation de crise.

7 7. Le module de gestion des utilisateurs et de sécurité (RBAC) : la confiance mais la vérification

7.1 Le problème : la sécurité du SIEM lui-même

Le SIEM contient des données extrêmement sensibles : logs d'authentification, données de vulnérabilités, informations sur les incidents. Un accès non autorisé à ces données serait catastrophique. Les premiers SIEM avaient des contrôles d'accès rudimentaires (un seul compte administrateur).

7.2 La solution : le modèle RBAC et l'audit

— **RBAC (Role-Based Access Control)** : définir des rôles (Lecteur, Analyste, Administrateur) avec des permissions granulaires.

— **Journalisation des actions** : chaque consultation, modification, suppression est enregistrée avec horodatage et identifiant utilisateur.

— **Ségrégation des accès** : par équipe, service, ou environnement (prod vs dev).

7.2.1 Pourquoi ce choix ?

Les exigences réglementaires (RGPD, HIPAA, ISO 27001) imposent une traçabilité et une limitation des accès. De plus, en cas de compromission d'un compte utilisateur, les logs d'audit permettent de retracer les actions malveillantes.

8 8. L'analyse comportementale (UEBA) : au-delà des règles

8.1 Le problème : les menaces internes et les comptes compromis

Les règles de corrélation ne peuvent pas détecter un utilisateur légitime qui abuse de ses droits (ex : un employé qui télécharge des données sensibles la nuit) ou un attaquant qui utilise des identifiants volés (activité "légitime" du point de vue du système). Il s'agit de menaces sans signature.

8.2 La solution : le profilage comportemental

L'UEBA (User and Entity Behavior Analytics) utilise l'apprentissage automatique pour établir une baseline du comportement normal de chaque utilisateur et de chaque machine (horaires de connexion, volumes de données, applications utilisées, etc.). Il détecte ensuite les anomalies.

8.2.1 Pourquoi ce choix ?

L'UEBA comble une faille fondamentale : il détecte ce qui est *anormal* plutôt que ce qui est *mal*. Il ne nécessite pas de signatures préexistantes. Il est particulièrement efficace pour détecter les menaces internes et les comptes compromis. Le *scoring de risque* permet de prioriser les investigations.

8.3 Les défis de l'UEBA

L'UEBA génère des faux positifs si la baseline n'est pas bien établie (ex : un nouvel utilisateur, un changement d'habitude). Il nécessite une phase d'apprentissage et une supervision humaine pour affiner les modèles. De plus, il est plus complexe à interpréter qu'une règle simple, ce qui peut rendre les analystes méfiants.

9 Conclusion : une évolution vers des architectures composables

L'architecture des SIEM que nous connaissons est le résultat de décennies d'adaptation aux défis du terrain. Chaque module a été ajouté pour résoudre une faille des modules précédents. L'agent de collecte a résolu les problèmes de performance et d'accès aux données. La normalisation a permis de parler un langage commun. La corrélation a automatisé l'intelligence de l'analyste. Le stockage distribué a permis de gérer le volume. Le SOAR a réduit le temps de réponse. L'UEBA a permis de détecter l'inconnu.

Aujourd'hui, une nouvelle tendance émerge : le **SIEM composable**. Au lieu d'une plateforme monolithique, les organisations assemblent des composants best-of-breed (collecteurs, moteurs de corrélation, bases de données, interfaces) via des API ouvertes et des schémas standardisés (comme ECS). Cette approche offre plus de flexibilité et évite le verrouillage, mais elle nécessite une maîtrise technique plus grande. C'est dans ce contexte que le projet *Smart SIEM* s'inscrit, en proposant une architecture modulaire, ouverte et adaptable, fidèle à l'esprit des systèmes qui ont fait leurs preuves.